

Playlist	Generative AI using LangChain – CampusX
Topics Covered	19 Video Modules (Intro → Agents)
Level	Beginner to Advanced
Best For	Interviews, Viva, Quick Revision

Table of Contents

1. Introduction to LangChain
2. LangChain Components
3. LangChain Models
4. Prompts in LangChain
5. Structured Output in LangChain
6. Output Parsers in LangChain
7. Chains in LangChain
8. Runnables in LangChain (Part 1 & 2)
9. Document Loaders
10. Text Splitters
11. Vector Stores
12. Retrievers in LangChain
13. Retrieval Augmented Generation (RAG)
14. Building a RAG Chatbot (YouTube Chatbot Project)
15. Tools in LangChain
16. Tool Calling in LangChain
17. Building End-to-End AI Agents
18. Interview Q&A; — Quick Fire Round

1. Introduction to LangChain

What is LangChain?

LangChain is an open-source Python (and JS) framework that makes it easy to build applications powered by Large Language Models (LLMs). It provides a standardized interface for connecting LLMs with data sources, memory, tools, and other components.

Created by **Harrison Chase** in Oct 2022. Written in Python and TypeScript.

Why LangChain?

Without LangChain you would need to:

- ✗ Write raw API calls for every LLM provider separately
- ✗ Manually manage prompt formatting and history
- ✗ Build custom logic for chaining multiple steps
- ✗ Handle vector DB integrations from scratch

LangChain solves this with:

- ✓ Unified interface across OpenAI, Anthropic, Gemini, Ollama etc.
- ✓ Ready-made chains, agents, and memory modules
- ✓ 100+ document loaders, vector store integrations
- ✓ LCEL (LangChain Expression Language) for composable pipelines

Key Use Cases

Use Case	Description
Chatbots	Conversational agents with memory
RAG Systems	LLM + your private documents
AI Agents	LLMs that use tools & take actions
Data Analysis	Query structured/unstructured data via LLM
Code Assistants	LLM + code tools

2. LangChain Components

The 6 Core Components

Models	Interface to LLMs (ChatOpenAI, Ollama, etc.)
Prompts	PromptTemplates to structure inputs to LLMs
Chains	Sequences that connect prompts + models + parsers
Memory	Store and retrieve conversation history
Indexes	Document loaders, text splitters, vector stores, retrievers
Agents	LLMs that decide which tools to call and when

LangChain Architecture (Mental Model)

User Input
↓ PromptTemplate — formats the input
↓ LLM / ChatModel — generates response
↓ OutputParser — structures the response
↓ Final Output (string / JSON / object)

3. LangChain Models

Types of Models in LangChain

LLM (Legacy)	Takes string input → returns string output. (e.g., OpenAI text-davinci)
ChatModel (Standard)	Takes list of messages → returns AIMessage. (e.g., ChatOpenAI, ChatGoogleGenerativeAI)
Embedding Models	Converts text to vectors for semantic search. (e.g., OpenAIEmbeddings, HuggingFaceEmbeddings)

Code — Using ChatOpenAI

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage

llm = ChatOpenAI(model='gpt-4o', temperature=0.7)

messages = [
    SystemMessage(content='You are a helpful assistant.'),
    HumanMessage(content='Explain RAG in 2 lines.')
]

response = llm.invoke(messages)
print(response.content)
```

Code — Using Ollama (Local LLM)

```
from langchain_ollama import ChatOllama

llm = ChatOllama(model='llama3')
response = llm.invoke('What is LangChain?')
print(response.content)
```

Code — Embedding Model

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
vector = embeddings.embed_query('LangChain is awesome')
print(len(vector)) # 1536 dimensions
```

Key Model Parameters

Parameter	Type	Purpose
temperature	float 0-2	Randomness of output. 0=deterministic, 1=creative
max_tokens	int	Maximum length of the generated response

model	str	Which model version to use (gpt-4o, llama3, etc.)
streaming	bool	Stream output token by token
top_p	float	Nucleus sampling — controls diversity

4. Prompts in LangChain

What is a PromptTemplate?

A **PromptTemplate** is a reusable, parameterized prompt. Instead of hardcoding prompts, you define a template with placeholders (variables) that get filled at runtime. This makes prompts modular, testable, and maintainable.

Types of Prompt Templates

PromptTemplate	For single-string (LLM) prompts with {variables}
ChatPromptTemplate	For chat models — list of SystemMessage, HumanMessage, AIMessage
FewShotPromptTemplate	Include example input-output pairs to guide the LLM
MessagesPlaceholder	Dynamically insert a list of messages (e.g., chat history)

Code — PromptTemplate

```
from langchain_core.prompts import PromptTemplate

template = PromptTemplate(
    input_variables=['topic', 'level'],
    template='Explain {topic} to a {level} student in simple terms.'
)

prompt = template.invoke({'topic': 'RAG', 'level': 'undergraduate'})
print(prompt.text)
```

Code — ChatPromptTemplate

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are an expert in {domain}.'),
    ('human', '{question}')
])

chain = prompt | llm
result = chain.invoke({'domain': 'NLP', 'question': 'What is tokenization?'})
print(result.content)
```

Code — FewShotPromptTemplate

```
from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate

examples = [
    {'input': 'happy', 'output': 'sad'},
```

```
{'input': 'fast', 'output': 'slow'},
]
example_prompt = PromptTemplate(
    input_variables=['input','output'],
    template='Input: {input}\nOutput: {output}'
)
few_shot = FewShotPromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
    suffix='Input: {word}\nOutput:',
    input_variables=['word']
)
print(few_shot.format(word='cold'))
```

5. Structured Output in LangChain

What is Structured Output?

Structured Output forces the LLM to respond in a defined schema (like JSON or a Pydantic model) instead of free-form text. This is essential for building reliable pipelines where downstream code needs to parse the LLM response programmatically.

Method: `.with_structured_output(schema)` — the cleanest approach in modern LangChain.

Code — `with_structured_output` (Pydantic)

```
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field

class Person(BaseModel):
    name: str = Field(description='Person name')
    age: int = Field(description='Age in years')
    city: str = Field(description='City of residence')

llm = ChatOpenAI(model='gpt-4o')
structured_llm = llm.with_structured_output(Person)

result = structured_llm.invoke('John is 28 years old and lives in Delhi.')
print(result.name, result.age, result.city)

# Output: John 28 Delhi
```

Code — `with_structured_output` (TypedDict)

```
from typing import TypedDict
from langchain_openai import ChatOpenAI

class Movie(TypedDict):
    title: str
    genre: str
    rating: float

structured_llm = ChatOpenAI(model='gpt-4o').with_structured_output(Movie)
result = structured_llm.invoke('Suggest a sci-fi movie.')
print(result) # {'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6}
```

Why Pydantic over TypedDict?

Pydantic (Recommended)

- Automatic type validation
- Rich Field descriptions for better prompting
- Nested models supported
- Default values & custom validators

TypedDict

- No runtime validation
- Simpler syntax for flat schemas
- Good for quick prototypes
- Less overhead

6. Output Parsers in LangChain

What are Output Parsers?

Output Parsers transform raw LLM output (string) into structured Python objects. They sit at the end of a chain and convert text → list, dict, Pydantic model, etc. Note: For modern LangChain, prefer `.with_structured_output()` for chat models. Output parsers are still useful for older patterns or non-chat LLMs.

Key Output Parsers

Parser	Output Type	Use When
StrOutputParser	str	Just want clean string output
JsonOutputParser	dict	LLM returns JSON string
PydanticOutputParser	Pydantic model	Need validated structured output
CommaSeparatedListOutputParser	list[str]	LLM returns comma-separated list
StructuredOutputParser	dict	Define schema via ResponseSchema
DatetimeOutputParser	datetime	Parse date/time strings

Code — StrOutputParser

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate.from_template('Tell me a joke about {topic}')
llm = ChatOpenAI()
parser = StrOutputParser()

chain = prompt | llm | parser

print(chain.invoke({'topic': 'Python'})) # Plain string output
```

Code — PydanticOutputParser

```
from langchain.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field

class Review(BaseModel):
    sentiment: str = Field(description='positive/negative/neutral')
    score: int = Field(description='Score out of 10')

parser = PydanticOutputParser(pydantic_object=Review)
# parser.get_format_instructions() gives the prompt instructions

prompt = ChatPromptTemplate.from_messages([
    ('system', 'Extract review info. {format_instructions}'),
```

```
('human', '{review}')  
]).partial(format_instructions=parser.get_format_instructions())  
  
chain = prompt | llm | parser  
result = chain.invoke({'review': 'Amazing product! 9/10'})  
print(result.sentiment, result.score)
```

7. Chains in LangChain

What is a Chain?

A **Chain** is a sequence of steps where the output of one step becomes the input of the next. Modern LangChain uses **LCEL (LangChain Expression Language)** with the pipe operator `|` to compose chains. This replaces the older LLMChain, SequentialChain patterns.

The Pipe Operator — LCEL

```
# Basic chain pattern: prompt | llm | parser
chain = prompt | llm | parser

# This is equivalent to:
# result = parser.invoke(llm.invoke(prompt.invoke(input)))

# Invoke the chain
result = chain.invoke({'input': 'Hello World'})
```

Types of Chains

Simple Chain (LCEL)	prompt llm parser — single step pipeline
Sequential Chain	Chain where output of one chain feeds into the next
Conditional Chain (RunnableBranch)	Route to different chains based on input condition
Parallel Chain (RunnableParallel)	Run multiple chains simultaneously, merge results
RetrievalQA Chain	Classic RAG chain: retriever + prompt + llm

Code — Sequential Chain

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI()

# Step 1: Summarize
summarize_prompt = ChatPromptTemplate.from_template('Summarize: {text}')

# Step 2: Translate
translate_prompt = ChatPromptTemplate.from_template('Translate to Hindi: {summary}')

chain1 = summarize_prompt | llm | StrOutputParser()
chain2 = translate_prompt | llm | StrOutputParser()
```

```
# Connect: output of chain1 goes to 'summary' input of chain2
full_chain = chain1 | (lambda x: {'summary': x}) | chain2
print(full_chain.invoke({'text': 'LangChain is a framework for building AI apps.'}))
```

8. Runnables in LangChain (LCEL)

What is a Runnable?

A **Runnable** is the base abstraction in LCEL. Everything in LangChain (prompts, LLMs, parsers, retrievers, tools) implements the Runnable interface. This means they all share a common API: `.invoke()`, `.stream()`, `.batch()`, `.ainvoke()`, `.astream()`.

Core Runnable Methods

Method	Description
<code>.invoke(input)</code>	Synchronous single call — returns output
<code>.stream(input)</code>	Returns a generator that streams tokens/chunks
<code>.batch([inputs])</code>	Process multiple inputs in parallel
<code>.ainvoke(input)</code>	Async version of invoke
<code>.astream(input)</code>	Async streaming
<code>.abatch([inputs])</code>	Async batch processing

Key Runnable Primitives

RunnableLambda	Wrap any Python function as a Runnable
RunnableParallel	Run multiple runnables in parallel, combine outputs as dict
RunnablePassthrough	Pass input unchanged (useful for adding original input to chain output)
RunnableBranch	Conditional routing — route to different chains based on condition
RunnableWithMessageHistory	Add memory/chat history to any runnable

Code — RunnableParallel

```
from langchain_core.runnables import RunnableParallel

parallel = RunnableParallel({
    'joke': joke_chain,
    'poem': poem_chain,
    'summary': summary_chain
})

# All three chains run simultaneously on the same input
result = parallel.invoke({'topic': 'Artificial Intelligence'})
print(result['joke']) # Output from joke_chain
```

```
print(result['poem']) # Output from poem_chain
```

Code — RunnablePassthrough

```
from langchain_core.runnables import RunnablePassthrough

# Pass the original question through while also running the retriever
chain = RunnableParallel({
    'context': retriever,
    'question': RunnablePassthrough() # keeps original input
}) | prompt | llm | StrOutputParser()

result = chain.invoke('What is RAG?')
```

9. Document Loaders

What are Document Loaders?

Document Loaders are components that load data from various sources (PDFs, URLs, CSVs, YouTube, Google Drive, Notion, etc.) and convert them into **Document objects**. A Document has two fields: **page_content** (the text) and **metadata** (source info).

Common Document Loaders

Loader Class	Source	Install Package
PyPDFLoader	PDF files	langchain-community
WebBaseLoader	Web URLs	langchain-community
TextLoader	Plain .txt files	Built-in
CSVLoader	CSV files	langchain-community
YoutubeLoader	YouTube transcripts	youtube-transcript-api
WikipediaLoader	Wikipedia articles	wikipedia
DirectoryLoader	Entire folder of files	Built-in
UnstructuredFileLoader	Any file type	unstructured

Code — Document Loaders

```
# PDF Loader

from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader('research_paper.pdf')
docs = loader.load() # returns List[Document]
print(docs[0].page_content[:200])
print(docs[0].metadata) # {'source': 'research_paper.pdf', 'page': 0}

# Web Loader

from langchain_community.document_loaders import WebBaseLoader

loader = WebBaseLoader('https://en.wikipedia.org/wiki/LangChain')
docs = loader.load()

# YouTube Loader

from langchain_community.document_loaders import YoutubeLoader

loader = YoutubeLoader.from_youtube_url(
    'https://www.youtube.com/watch?v=...',
    add_video_info=True
)

docs = loader.load()
```

10. Text Splitters

Why Text Splitting?

LLMs have a **context window limit** (e.g., 128K tokens for GPT-4o). Large documents cannot fit entirely. We split documents into smaller **chunks** and store them in a vector DB. At query time, only the relevant chunks are retrieved.

Key parameters: `chunk_size` (tokens/chars per chunk), `chunk_overlap` (overlap between chunks to preserve context)

Types of Text Splitters

CharacterTextSplitter	Splits on a single character (default: '\n\n'). Simple and fast.
RecursiveCharacterTextSplitter	RECOMMENDED. Tries to split on ['\n\n','n',' ',''] in order. Preserves semantic units.
TokenTextSplitter	Splits based on actual token count (important for precise context window management).
MarkdownHeaderTextSplitter	Splits Markdown documents based on header levels (#, ##, ###).
HTMLHeaderTextSplitter	Splits HTML documents at heading tags (h1, h2, h3).
SemanticChunker	Uses embeddings to find natural semantic break points in text.

Code — RecursiveCharacterTextSplitter

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, # Characters per chunk
    chunk_overlap=50, # Overlap to maintain context
    separators=['\n\n', '\n', ' ', ''] # Order of splitting
)

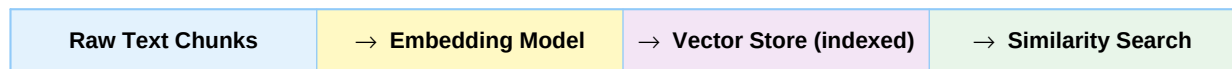
# Split documents from a loader
docs = loader.load()
chunks = splitter.split_documents(docs)
print(f'Total chunks: {len(chunks)}')
print(chunks[0].page_content)
print(chunks[0].metadata)
```

11. Vector Stores

What is a Vector Store?

A **Vector Store (Vector Database)** stores text chunks as high-dimensional vectors (embeddings) and supports fast **similarity search**. When a query comes in, it is converted to a vector and the most similar vectors (chunks) are retrieved using cosine similarity or dot product.

Flow: Documents → Embeddings → Vector Store



Popular Vector Stores

Vector Store	Type	Best For
FAISS	In-memory	Quick prototypes, no server needed
Chroma	Local / persistent	Development, small datasets
Pinecone	Cloud managed	Production, large scale
Weaviate	Open source cloud	Production with filtering
Qdrant	Open source	High performance production
PGVector	PostgreSQL ext.	When already using PostgreSQL

Code — FAISS Vector Store

```
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()

# Create from documents
vectorstore = FAISS.from_documents(chunks, embeddings)

# Save locally
vectorstore.save_local('faiss_index')

# Load later
vectorstore = FAISS.load_local('faiss_index', embeddings,
allow_dangerous_deserialization=True)

# Similarity search
results = vectorstore.similarity_search('What is RAG?', k=3)
for doc in results:
    print(doc.page_content[:100])
```

12. Retrievers in LangChain

What is a Retriever?

A **Retriever** is an abstraction that takes a query string and returns relevant Documents. It wraps a vector store (or other search mechanism) with a simple **.invoke(query)** interface. All retrievers implement the Runnable interface and can be composed into chains.

Types of Retrievers

VectorStoreRetriever	Basic retriever from any vector store. Uses similarity search.
MultiQueryRetriever	Generates multiple paraphrases of the query via LLM, retrieves for each, merges results.
ContextualCompression Retriever	Compresses/filters retrieved docs to only the relevant parts.
BM25Retriever	Keyword-based TF-IDF search. No embeddings needed. Good for exact terms.
EnsembleRetriever	Combines multiple retrievers (e.g., BM25 + vector) using Reciprocal Rank Fusion.
SelfQueryRetriever	LLM parses the user query and creates structured metadata filters automatically.
ParentDocumentRetriever	Retrieves small chunks but returns full parent documents.

Code — Basic Retriever

```
# From any vector store
retriever = vectorstore.as_retriever(
    search_type='similarity', # or 'mmr', 'similarity_score_threshold'
    search_kwargs={'k': 4} # Return top 4 documents
)

docs = retriever.invoke('What is attention mechanism?')
print(len(docs)) # 4
```

Code — MultiQueryRetriever

```
from langchain.retrievers import MultiQueryRetriever
from langchain_openai import ChatOpenAI

llm = ChatOpenAI()
retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(),
    llm=llm
)
```

```
# Internally generates 3+ query variations, retrieves for each
docs = retriever.invoke('Tell me about transformers')
```

Code — EnsembleRetriever (Hybrid Search)

```
from langchain.retrievers import EnsembleRetriever, BM25Retriever

bm25 = BM25Retriever.from_documents(chunks, k=3)
vec = vectorstore.as_retriever(search_kwargs={'k': 3})

ensemble = EnsembleRetriever(
    retrievers=[bm25, vec],
    weights=[0.4, 0.6] # Blend keyword + semantic
)

docs = ensemble.invoke('transformer attention')
```

13. Retrieval Augmented Generation (RAG)

What is RAG?

RAG (Retrieval Augmented Generation) is an AI architecture that enhances LLM responses by retrieving relevant information from an external knowledge base BEFORE generating an answer. This solves LLM hallucination and knowledge cutoff problems by grounding responses in real documents.

RAG = **Retriever** (finds relevant chunks) + **Generator** (LLM that uses those chunks to answer)

RAG Pipeline — Two Phases

INDEXING PHASE (Done Once / Offline)	QUERYING PHASE (Done Each Time)
1. Load Documents (loaders) 2. Split into Chunks (splitters) 3. Embed chunks (embedding model) 4. Store in Vector DB	1. Embed user query 2. Retrieve top-k similar chunks 3. Build prompt with context + query 4. LLM generates grounded answer

Code — Complete RAG Pipeline

[illegible]

```
def format_docs(docs):
    return '\n\n'.join(d.page_content for d in docs)

rag_chain = (
    {'context': retriever | format_docs, 'question': RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)

answer = rag_chain.invoke('What is the main conclusion of the paper?')
print(answer)
```

14. Building a RAG Chatbot — YouTube Chatbot Project

Project Overview

Build a chatbot that can answer questions about any YouTube video by extracting the transcript, indexing it in a vector store, and using RAG to generate answers grounded in the video content.

Tech Stack: LangChain + YoutubeLoader + FAISS + OpenAI + ChatPromptTemplate + LCEL

Code — YouTube RAG System

```
from langchain_community.document_loaders import YoutubeLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

YOUTUBE_URL = 'https://www.youtube.com/watch?v=YOUR_VIDEO_ID'

# Load YouTube transcript
loader = YoutubeLoader.from_youtube_url(YOUTUBE_URL, add_video_info=False)
docs = loader.load()

# Split & Index
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)
vectorstore = FAISS.from_documents(chunks, OpenAIEmbeddings())
retriever = vectorstore.as_retriever(search_kwargs={'k': 4})

# Prompt
prompt = ChatPromptTemplate.from_messages([
    ('system', 'Answer only from the video transcript.\nContext:\n{context}'),
    ('human', '{question}')
])

def format_docs(docs):
    return '\n'.join(d.page_content for d in docs)

chain = (
    {'context': retriever | format_docs, 'question': RunnablePassthrough()}
    | prompt | ChatOpenAI() | StrOutputParser()
)

# Chat loop
```

```
while True:
    q = input('You: ')
    if q.lower() == 'quit': break
    print('Bot:', chain.invoke(q))
```

15. Tools in LangChain

What are Tools?

Tools are functions that an LLM can call to interact with external systems or perform computations. Instead of just generating text, an LLM with tools can: search the web, run Python code, query databases, call APIs, read files, and more.

Tools are the bridge between the LLM's reasoning and real-world actions.

Creating Tools — Three Methods

Method 1: @tool decorator (Simplest)

```
from langchain_core.tools import tool

@tool
def multiply(a: int, b: int) -> int:
    '''Multiply two integers together.'''
    return a * b

print(multiply.name) # 'multiply'
print(multiply.description) # 'Multiply two integers together.'
print(multiply.args) # {'a': {...}, 'b': {...}}
```

Method 2: StructuredTool (More control)

```
from langchain_core.tools import StructuredTool
from pydantic import BaseModel

class AddInput(BaseModel):
    a: int
    b: int

def add(a: int, b: int) -> int:
    return a + b

add_tool = StructuredTool.from_function(
    func=add,
    name='add',
    description='Add two numbers',
    args_schema=AddInput
)
```

Built-in LangChain Tools

Tool	Package	What it does
------	---------	--------------

TavilySearchResults	langchain-community	Web search via Tavily API
WikipediaQueryRun	langchain-community	Query Wikipedia
PythonREPLTool	langchain-experimental	Run Python code
DuckDuckGoSearchRun	langchain-community	Free web search
ArxivQueryRun	langchain-community	Search academic papers

16. Tool Calling in LangChain

What is Tool Calling?

Tool Calling (also called Function Calling) is when an LLM decides WHICH tool to call and with WHAT arguments based on the user's request. The LLM returns a structured tool call specification, which is then executed, and the result is fed back to the LLM.

Key difference: Tools are *defined* separately; Tool Calling is the *mechanism* by which the LLM invokes them.

Tool Calling Flow

Step	What Happens
1. Bind Tools	<code>llm.bind_tools([tool1, tool2])</code> — tell LLM what tools exist
2. LLM Decides	LLM returns <code>AIMessage</code> with <code>tool_calls</code> field (not final answer)
3. Execute Tool	Extract tool name + args, call the actual function
4. Return Result	Add <code>ToolMessage</code> with result back to conversation
5. LLM Responds	LLM uses tool result to generate final human-readable answer

Code — Tool Calling End-to-End

```
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage, ToolMessage

@tool
def get_weather(city: str) -> str:
    '''Get weather for a city.'''
    return f'Weather in {city}: Sunny, 32 degrees C'

@tool
def calculate(expression: str) -> float:
    '''Evaluate a math expression.'''
    return eval(expression)

tools = [get_weather, calculate]
llm = ChatOpenAI(model='gpt-4o').bind_tools(tools)

messages = [HumanMessage('What is weather in Delhi and what is 25 * 4?')]
response = llm.invoke(messages)

# LLM returns tool calls, not a final answer
for tc in response.tool_calls:
    tool_name = tc['name']
```

```
tool_args = tc['args']

# Execute the tool

tool_fn = {'get_weather': get_weather, 'calculate': calculate}[tool_name]
result = tool_fn.invoke(tool_args)
messages.append(ToolMessage(content=str(result), tool_call_id=tc['id']))

# Final LLM response with tool results
final = llm.invoke(messages)
print(final.content)
```

17. Building End-to-End AI Agents

What is an AI Agent?

An **AI Agent** is an LLM that can **reason** → **decide** → **act** in a loop. Unlike a fixed chain, an agent can choose which tools to call, in which order, based on intermediate results. The agent keeps running until it reaches a final answer.

ReAct Pattern: Reasoning + Acting — agent thinks step-by-step and interleaves reasoning traces with tool calls.

Agent Types in LangChain

ReAct Agent	Most common. Uses Thought → Action → Observation loop. Works with any LLM.
OpenAI Functions Agent	Uses OpenAI's native function calling. More reliable for GPT models.
OpenAI Tools Agent	Updated version of Functions agent using the tools API.
Structured Chat Agent	For multi-input tools. Good for complex tool schemas.
Self-Ask with Search	Agent that asks itself sub-questions and searches to answer each.

Code — ReAct Agent with `create_react_agent`

```
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain import hub
from langchain.agents import create_react_agent, AgentExecutor

@tool
def search_wikipedia(query: str) -> str:
    '''Search Wikipedia for information.'''
    from wikipedia import summary
    return summary(query, sentences=3)

@tool
def calculator(expression: str) -> str:
    '''Calculate mathematical expressions.'''
    return str(eval(expression))

tools = [search_wikipedia, calculator]
llm = ChatOpenAI(model='gpt-4o', temperature=0)

# Pull standard ReAct prompt from LangChain Hub
prompt = hub.pull('hwchase17/react')
```

```
agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)
executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True,
    max_iterations=5
)

result = executor.invoke({
    'input': 'Who invented Python and what year? Also calculate 1991 + 33.'
})

print(result['output'])
```

Agent vs Chain — Key Difference

Chain

- Fixed execution flow
- Steps decided at design time
- Deterministic path
- No intermediate decisions
- Good for: pipelines with known steps

Agent

- Dynamic execution
- LLM decides next step at runtime
- Can loop and retry
- Handles unexpected situations
- Good for: open-ended tasks, research

18. Interview Q&A; — Quick Fire Round

Q: What is LangChain and why is it used?

A: LangChain is an open-source framework for building LLM-powered applications. It provides abstractions for prompts, models, chains, memory, agents and integrations — saving developers from writing boilerplate code.

Q: What is LCEL?

A: LCEL (LangChain Expression Language) is a declarative way to compose Runnable components using the pipe operator `|`. It enables streaming, async, batching, and parallelism automatically. e.g., `chain = prompt | llm | parser`

Q: Difference between LLM and ChatModel in LangChain?

A: LLM takes a string and returns a string (legacy). ChatModel takes a list of messages (HumanMessage, SystemMessage, AIMessage) and returns an AIMessage. Modern LangChain almost always uses ChatModel.

Q: What is a PromptTemplate vs ChatPromptTemplate?

A: PromptTemplate is for single string prompts (legacy LLMs). ChatPromptTemplate structures prompts as a list of role-based messages (system, human, ai) for chat models.

Q: What is with_structured_output()?

A: It's a method on chat models that forces the LLM to return output conforming to a Pydantic schema or TypedDict. Internally uses function calling or JSON mode. Preferred over PydanticOutputParser for modern LangChain.

Q: What is RAG and why is it needed?

A: RAG (Retrieval Augmented Generation) combines a retriever (vector search) with an LLM. It solves the knowledge cutoff problem and reduces hallucinations by grounding LLM responses in retrieved real documents.

Q: What is the difference between a Retriever and a Vector Store?

A: A Vector Store stores embeddings and supports similarity search. A Retriever is an abstraction that wraps a vector store with a simple `.invoke(query) → List[Document]` interface, making it composable in LCEL chains.

Q: What is chunk_overlap in text splitting and why is it needed?

A: `chunk_overlap` is the number of characters/tokens shared between consecutive chunks. It ensures that context at chunk boundaries is not lost — information that spans two chunks is still captured in at least one of them.

Q: What is a Tool in LangChain?

A: A Tool is a Python function wrapped with metadata (name, description, args schema) so an LLM can decide when and how to call it. Created with `@tool` decorator or `StructuredTool.from_function()`.

Q: What is the difference between Tool and Tool Calling?

A: A Tool is the function definition. Tool Calling is the mechanism where the LLM (when given a list of tools) generates structured tool call arguments instead of plain text, and those are then executed.

Q: What is a ReAct Agent?

A: ReAct (Reasoning + Acting) is an agent that alternates between Thought (reasoning what to do), Action (calling a tool), and Observation (seeing the result), until it can give a final answer.

Q: What is RunnablePassthrough?

A: A Runnable that passes its input unchanged to the next step. Useful in RunnableParallel to preserve the original user question while other branches process it (e.g., retriever branch).

Q: What is RunnableParallel?

A: A Runnable that runs multiple runnables concurrently on the same input and returns their outputs as a dict. Used to build the {context, question} dict for RAG chains.

Q: What is the difference between .invoke(), .batch(), and .stream()?

A: .invoke() processes a single input synchronously. .batch() processes a list of inputs in parallel. .stream() returns a generator yielding output chunks token by token for real-time display.

Q: What is a MultiQueryRetriever?

A: It uses an LLM to generate multiple paraphrased versions of the user's query, retrieves documents for each, deduplicates, and returns a richer set of results — improving recall.

Q: What is an EnsembleRetriever?

A: Combines multiple retrievers (e.g., BM25 keyword + FAISS vector) using Reciprocal Rank Fusion to merge their ranked results. Provides hybrid search — best of both keyword and semantic matching.

Q: What is the difference between FAISS and Chroma?

A: FAISS is an in-memory vector store by Meta — fast, no persistence by default, great for prototypes. Chroma is a local vector database that supports persistent storage and metadata filtering, better for development.

Q: How does memory work in LangChain?

A: Memory stores conversation history. RunnableWithMessageHistory wraps any chain to automatically load and save chat history from a storage backend. The history is injected into the prompt via MessagesPlaceholder.

Q: What is Few-Shot Prompting?

A: Providing the LLM with example input-output pairs in the prompt to guide it toward the desired output format or reasoning style, without fine-tuning the model weights.

Q: What is the difference between an Agent and a Chain?

A: A Chain has a fixed, predetermined execution path. An Agent uses the LLM to dynamically decide which tool to call next based on intermediate results — it can loop, branch, and adapt at runtime.

Quick Reference — Import Cheat Sheet

[illegible]